MONASH University

# When Constraints Break

## Week 4 — Hill Climbing

Julian Garcia

FIT1061 — Introduction to Artificial Intelligence

COMMONWEALTH OF AUSTRALIA | Copyright Regulations 1969 | WARNING: This material has been reproduced and communicated to you by or on behalf of Monash University pursuant to Part VB of the Copyright Act 1968 (the Act). The material in this communication may be subject to copyright under the Act. Any further reproduction or communication of this material by you may be the subject of copyright protection under the Act. Do not remove this notice.


RD GATE STATUS
CANCELLED
CANCELLED
CANCELLED
CANCELLED
CANCELLED
CANCELLED

Photo: AP

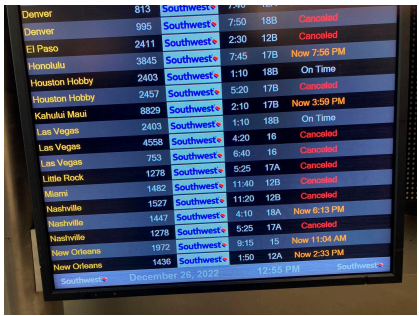
MILESTONE

# Southwest Airlines Collapse

December 21–30, 2022

*When the schedule couldn’t reschedule itself.*

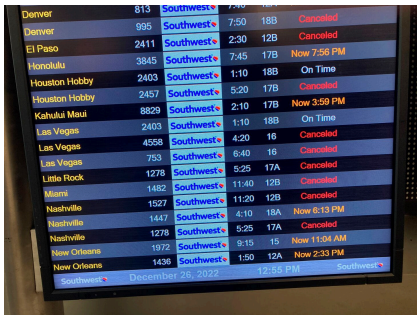
# What happened?




Late December 2022, US airport.

A winter storm hit on December 21. Southwest cancelled more flights than every other US carrier — **combined**.

# What happened?

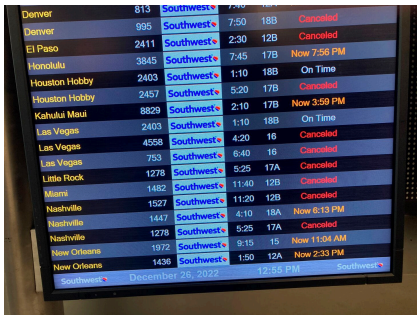



Late December 2022, US airport.

A winter storm hit on December 21. Southwest cancelled more flights than every other US carrier — **combined**.

Over **16,700** flights cancelled. More than **2 million** passengers affected.

# What happened?




Late December 2022, US airport.

A winter storm hit on December 21. Southwest cancelled more flights than every other US carrier — **combined**.

Over **16,700** flights cancelled. More than **2 million** passengers affected.

Other airlines recovered within 2–3 days. Southwest took **10**.

# Poll

Two airlines are hit by the same storm. One sends most flights through a few big hub airports; the other flies many direct city-to-city routes.

## Which one recovers faster?

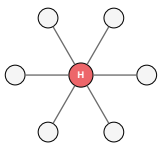
- **The hub airline**
- **The direct-route airline**
- **About the same**

**Vote now** — scan the code for your campus, *or* type the address.

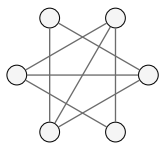
**Melbourne**
pollev.com/juliangarcia

**Malaysia**
pollev.com/onghueyfang767

# Why Southwest, why so long?



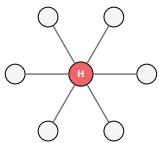

Hub-and-spoke



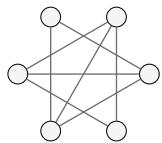
Point-to-point

Hub-and-spoke → delay stays **contained**.

# Why Southwest, why so long?




Hub-and-spoke



Point-to-point

Hub-and-spoke → delay stays **contained**.

Point-to-point → delay **cascades**: plane, then crew, then the next flight segment.

**Cascading constraints.**

Why couldn’t the **scheduling algorithm** recover?

# Today

Local search

Hill climbing

Local minima

Random restarts

At scale

In the lab

Local search
Hill climbing
Local minima
Random restarts
At scale
In the lab

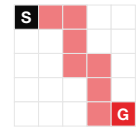
# Local search

*when you can’t afford to keep track*

# So far, one kind of question




Last week: **BFS / DFS**.




This week so far: **greedy best-first**.

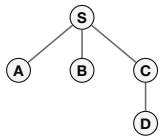
Both asking **the same question**: *what is the path from A to B?*

# Some problems don’t have a path

| The problem | The answer is | Path matters? |
|---|---|---|
| Your exam timetable | a **schedule** | no |
| Sudoku | a **filled grid** | no |
| Southwest’s flight + crew assignments | a **schedule** | no |

# What were we keeping track of?

**Journey matters → frontier holds journeys**




```
frontier = [S→A, S→B, S→C→D]
```

**Configuration matters → no journey to hold**

| | | | | | | | | |
|---|---|---|---|---|---|---|---|---|
| 5 | 3 | 4 | 6 | 7 | 8 | 9 | 1 | 3 |
| 6 | 7 | 2 | 1 | 9 | 5 | 3 | 4 | 8 |
| 1 | 9 | 8 | 3 | 4 | 2 | 5 | 6 | 7 |
| 8 | 5 | 9 | 7 | 6 | 1 | 4 | 2 | 3 |
| 4 | 2 | 6 | 8 | 2 | 3 | 7 | 9 | 1 |
| 7 | 1 | 3 | 9 | 2 | 4 | 8 | 5 | 6 |
| 9 | 6 | 1 | 5 | 3 | 7 | 2 | 8 | 4 |
| 2 | 8 | 7 | 4 | 1 | 9 | 6 | 3 | 5 |
| 3 | 4 | 5 | 2 | 8 | 6 | 1 | 7 | 9 |

```
current = grid
```
(**red** = conflicts)

If the journey doesn't matter, we **don't need the frontier here**.

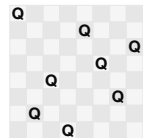
# A name for this family

**Constraint satisfaction problem**

A set of **variables**, each with a **domain** of possible values, plus a set of **constraints** that say which combinations are legal.

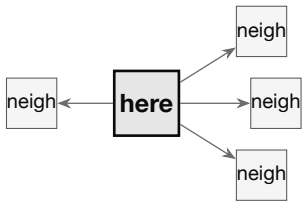



Variables = regions. Domain = {red, grey, black}.
Constraint = neighbours differ.




Variables = the 8 rows. Domain = column for each row. Constraint = no two attack.

# The local-search recipe

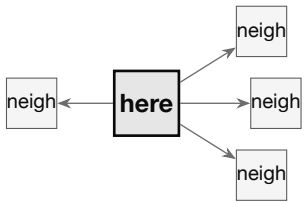



We stand at one state, score it, then look at its neighbours.

**local_search**

```
function LOCAL_SEARCH(start):
    current ← start
    while true:
        if no neighbour is strictly better:
            return current        // stuck
        current ← best neighbour
```

# The local-search recipe




We stand at one state, score it, then look at its neighbours.

**local_search**

```
function LOCAL_SEARCH(start):
    current ← start
    while true:
        if no neighbour is strictly better:
            return current        // stuck
        current ← best neighbour
```

BFS and DFS grew a frontier. Local search keeps **one state**.

So memory stays **flat** — however big the problem gets.

Local search

Hill climbing

Local minima

Random restarts

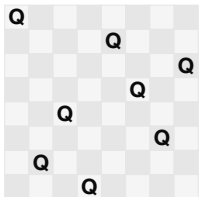
At scale

In the lab

# Hill climbing

*the simplest local search*

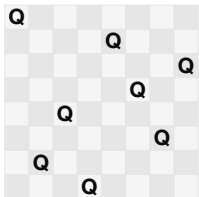
# The 8-queens problem




One of **92** valid solutions.

Place 8 queens on a chessboard so that **no two attack** — no shared row, column, or diagonal.

# The 8-queens problem

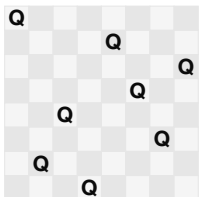



One of **92** valid solutions.

Place 8 queens on a chessboard so that **no two attack** — no shared row, column, or diagonal.

Posed 1848; **92** solutions for $n=8$.

# The 8-queens problem




One of **92** valid solutions.

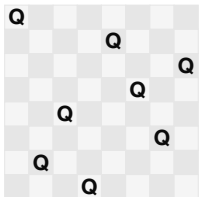
Place 8 queens on a chessboard so that **no two attack** — no shared row, column, or diagonal.

Posed 1848; **92** solutions for $n=8$.

The **smallest** problem with the same **shape** as the schedule that broke Southwest:

many constraints, and too many states to check one by one.

# The 8-queens problem




One of **92** valid solutions.

Place 8 queens on a chessboard so that **no two attack** — no shared row, column, or diagonal.

Posed 1848; **92** solutions for $n=8$.

The **smallest** problem with the same **shape** as the schedule that broke Southwest:

many constraints, and too many states to check one by one.

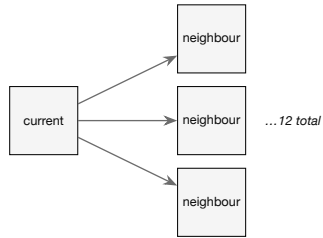
First we'll learn the moves on an even smaller board — 4 queens — then scale back up.

# Warm-up — 4 queens: state and neighbours




**State:** one queen per row — so the board is just 4 column choices.

**Violations:** count attacking pairs (here: **3**). This is the **score** we minimise.




**Neighbours:** slide *one queen* along its row to a different column.

Each queen has 3 other columns. $4 \times 3 = 12$ **neighbours** — few enough to check by hand.

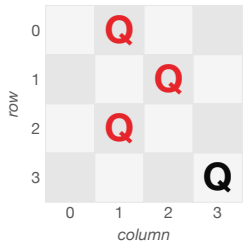
# Hill climbing

**hill_climbing**

```
function HILL_CLIMBING(board):
    current ← board
    while true:
        violations_here ← count_violations(current)
        neighbours      ← get_neighbours(current)
        best            ← argmin(neighbours by count_violations)
        if count_violations(best) ≥ violations_here:
            return current              // local minimum
        current ← best
```

- *Best* = fewest violations. *Strict* improvement, or stop.
- *Ties:* pick the first best neighbour we find.
- *Name:* called **hill climbing** by convention; for our problem it walks *downhill in violations*.
- *Returns:* a board — which may or may not be a valid solution.

# Which neighbour is best?




**Violations: 3**

- Pick any queen. Move it one column. **Better, worse, or same?**
- If we did this for all **12 neighbours**, which would we move to?

You'll run this same step in code in the lab.

# Poll

We run this method 100 times on the 8-queens puzzle, each time from a different random start.

**Out of 100 runs, how many reach a full solution?**

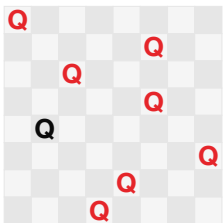
- **Almost all (about 90)**
- **About half**
- **Only a few (about 14)**

**Vote now** — scan the code for your campus, *or* type the address.

**Melbourne**
pollev.com/juliangarcia

**Malaysia**
pollev.com/onghueyfang767

# Now scale up: 8 queens

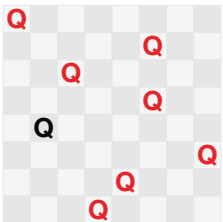


Same rules, same move — just **8** queens now.

$8 \times 7 = 56$ **neighbours** per step.

Now the space is huge: $8^8 \approx$ **17 million** states.

# Now scale up: 8 queens




Same rules, same move — just **8** queens now.

$8 \times 7 =$ **56 neighbours** per step.

Now the space is huge: $8^8 \approx$ **17 million** states.

4 queens was the warm-up — small enough to trace by hand.

8 queens is the realistic size.

Local search

Hill climbing

**Local minima**

Random restarts

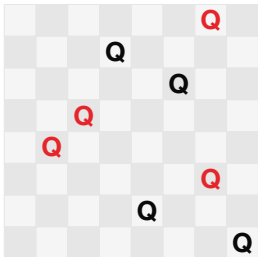
At scale

In the lab

# Local minima

*why it gets stuck*

# The board it got stuck on

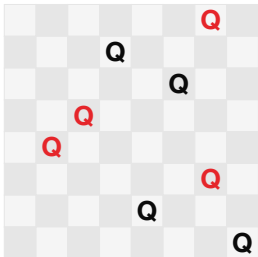



**Violations: 2**

The live run stopped here.

Violations fell $9 \to 6 \to 4 \to 3 \to 2$ — then nothing improved.

# The board it got stuck on



**Violations: 2**

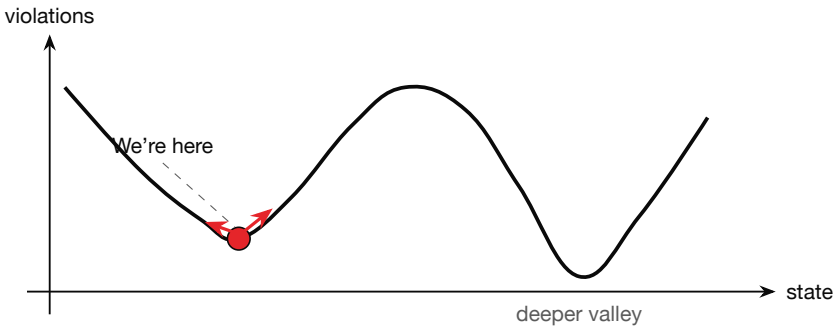
The live run stopped here.

Violations fell $9 \to 6 \to 4 \to 3 \to 2$ — then nothing improved.

Every one of the **56** neighbours has **2 or more**.

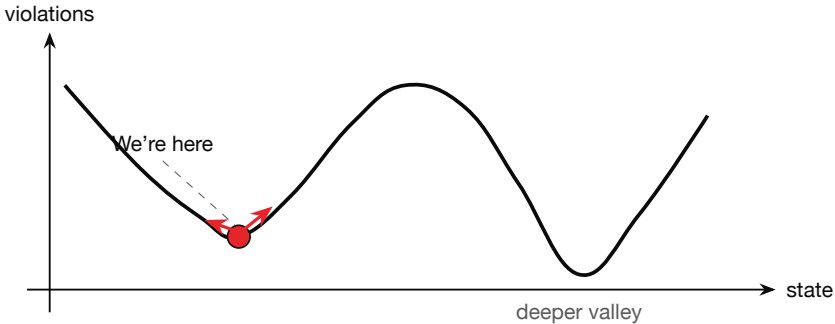
A **local minimum**: two pairs short of a solution, and hill climbing returns it.

# A 1D landscape




Both neighbours are higher. **Hill climbing stops.**

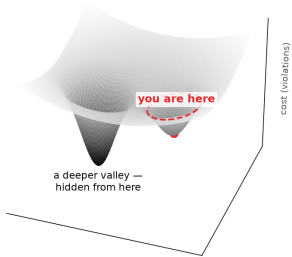
# A 1D landscape




Both neighbours are higher. **Hill climbing stops.**

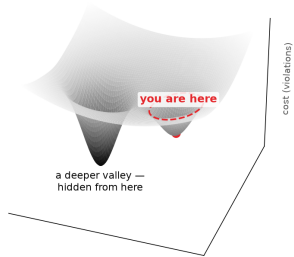
But the **deeper** valley is over there.

# A 2D landscape




From here, the shallow basin **looks like the best we can do**.

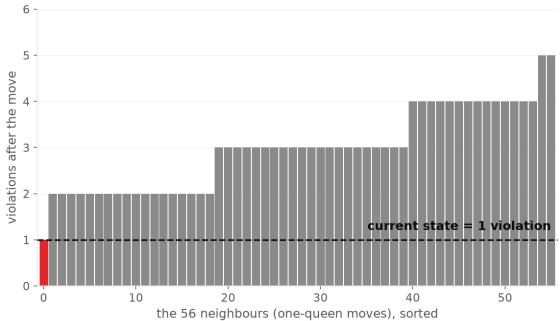
# A 2D landscape




From here, the shallow basin **looks like the best we can do**.

Hill climbing sees **only its neighbours**.

# The 8-queens landscape




8-queens has a landscape too.

Discrete, high-dimensional, **same geometry**.

The algorithm did exactly what we asked.

# The algorithm did exactly what we asked.

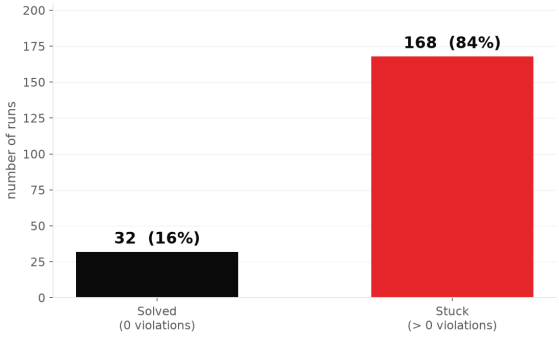
Always move to a strictly better neighbour. Otherwise stop.

# The algorithm did exactly what we asked.

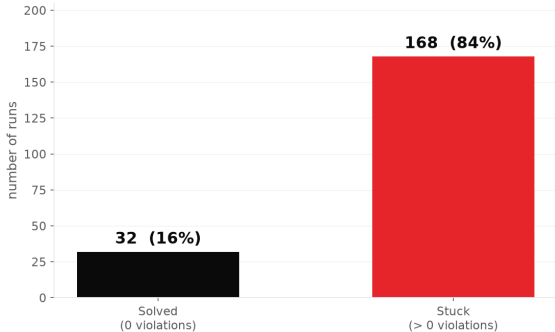
Always move to a strictly better neighbour. Otherwise stop.

**The problem has the local minima built in.**

# And the empirical price



# And the empirical price




Stuck ∼**84%** of the time — and it gives up after ∼**3 steps**.

*Russell & Norvig report 86%; our experiment is close.*

Local search

Hill climbing

Local minima

**Random restarts**

At scale

In the lab

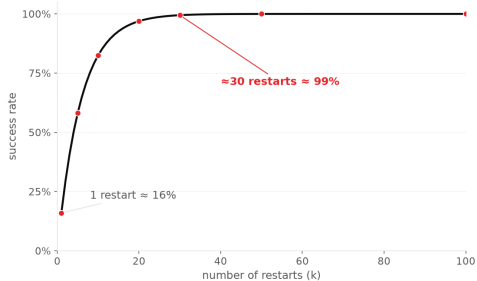
# Random restarts

# Random restarts

Stuck? **Start again from a new random board.**

**random_restarts**

```
function RANDOM_RESTARTS(k):
    for k attempts:
        board  ← random_start()
        result ← hill_climbing(board)
        if result is a solution:
            return result
    return best attempt seen
```




Local minima don't go away. **But we can keep trying from new random starts.**

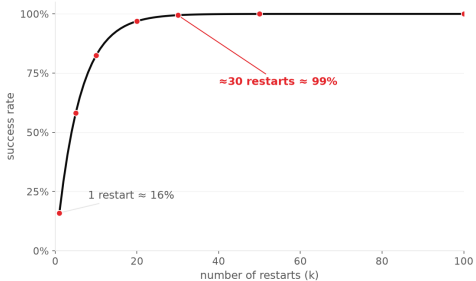
Restarts scale: a few tries usually crack 8-queens. With *smarter* move-picking, local search solves even **3,000,000 queens** in seconds.

# Random restarts

Stuck? **Start again from a new random board.**

**random_restarts**

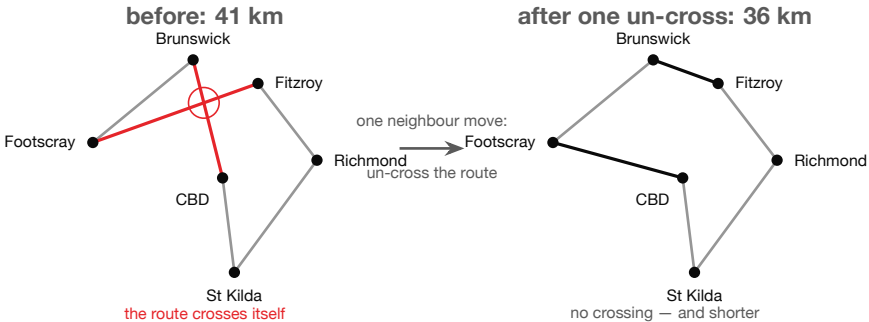
```
function RANDOM_RESTARTS(k):
    for k attempts:
        board  ← random_start()
        result ← hill_climbing(board)
        if result is a solution:
            return result
    return best attempt seen
```



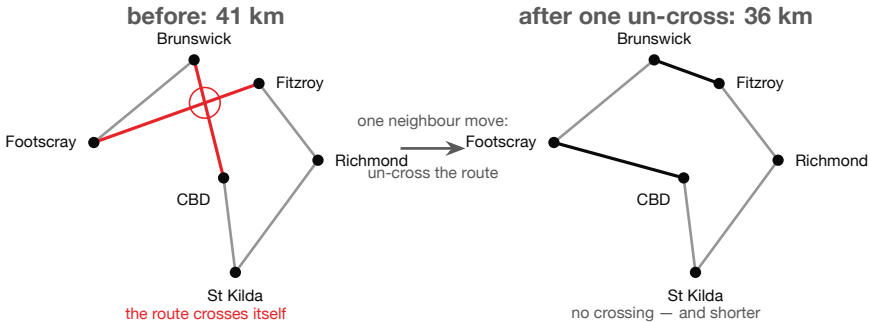

Local minima don't go away. **But we can keep trying from new random starts.**

Restarts scale: a few tries usually crack 8-queens. With *smarter* move-picking, local search solves even **3,000,000 queens** in seconds.

# The same recipe, on a map



# The same recipe, on a map

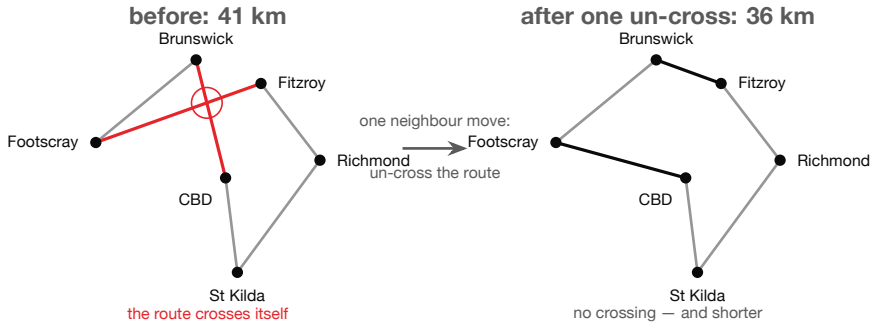



**State:** the whole route

**Score:** total distance

**Neighbour:** reverse a stretch of stops (un-cross the route)

# The same recipe, on a map




**State:** the whole route **Score:** total distance **Neighbour:** reverse a stretch of stops (un-cross the route)

The score can be **any quantity we minimise** — clashes on a board, kilometres on a map.

Local search

Hill climbing

Local minima

Random restarts

**At scale**

In the lab

# At scale

# You've been stuck here too

| | Mon | Tue | Wed | Thu |
|---|---|---|---|---|
| 9am | Maths | | Chem | |
| 1pm | Physics | Bio | CompSci | |
| 4pm | | Eng + Art | History | |

A full exam timetable is a **complete state** — every exam in a slot.

**Violations:** two of your exams in the same slot.

**Neighbour:** move one exam to another slot.

# You've been stuck here too

| | Mon | Tue | Wed | Thu |
|---|---|---|---|---|
| 9am | Maths | | Chem | |
| 1pm | Physics | Bio | CompSci | |
| 4pm | | **Eng + Art** | History | |

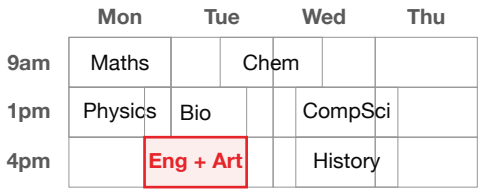
A full exam timetable is a **complete state** — every exam in a slot.

**Violations:** two of your exams in the same slot.

**Neighbour:** move one exam to another slot.

Move Art out of that slot — and it clashes somewhere else. **Every fix creates a new clash.**

# You've been stuck here too




A full exam timetable is a **complete state** — every exam in a slot.

**Violations:** two of your exams in the same slot.

**Neighbour:** move one exam to another slot.

Move Art out of that slot — and it clashes somewhere else. **Every fix creates a new clash.**

The **same problem** as the queens.

# The same shape, everywhere

| Where | A “state” is… | What we minimise |
|---|---|---|
| Delivery routes | the route through all the stops | total distance |
| Drilling circuit boards | the order to drill the holes | drill-head travel |
| Exam timetables | every exam placed in a slot | clashes |
| Sports fixtures | who plays whom, each round | travel + broken rules |
| **Airline scheduling** | **crews + planes on every flight** | **delays + broken rules** |

# The same shape, everywhere

| Where | A “state” is… | What we minimise |
|---|---|---|
| Delivery routes | the route through all the stops | total distance |
| Drilling circuit boards | the order to drill the holes | drill-head travel |
| Exam timetables | every exam placed in a slot | clashes |
| Sports fixtures | who plays whom, each round | travel + broken rules |
| **Airline scheduling** | **crews + planes on every flight** | **delays + broken rules** |

Every row: **a state, and a score to minimise.**

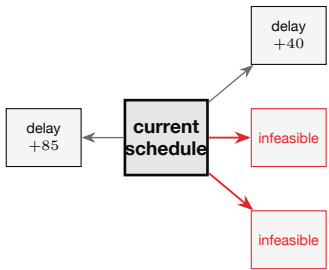
# The same shape, everywhere

| Where | A “state” is… | What we minimise |
|---|---|---|
| Delivery routes | the route through all the stops | total distance |
| Drilling circuit boards | the order to drill the holes | drill-head travel |
| Exam timetables | every exam placed in a slot | clashes |
| Sports fixtures | who plays whom, each round | travel + broken rules |
| **Airline scheduling** | **crews + planes on every flight** | **delays + broken rules** |

Every row: **a state, and a score to minimise.**

The red row broke, in December 2022.

# Back to Southwest — the same shape

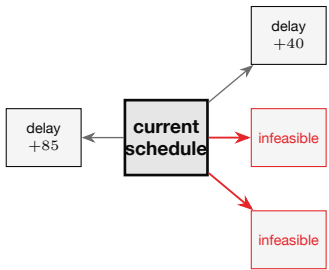



Southwest's crew-recovery system had to find a **new feasible schedule** from a heavily degraded current one.

**Score:** total delay + constraint violations.

**Neighbour:** reroute one plane, swap one crew.

# Back to Southwest — the same shape




Southwest's crew-recovery system had to find a **new feasible schedule** from a heavily degraded current one.

When 700+ flights cancel in a day, every nearby schedule hits a constraint — crew rest rules, aircraft position, gate availability. **Every nearby schedule is worse or against the rules.**

**Score:** total delay + constraint violations.

**Neighbour:** reroute one plane, swap one crew.

# The cost

| | |
|---|---|
| Flights cancelled (Dec 21–31) | ∼16,700 |
| Passengers affected | >2 million |
| Time to recover | 10 days |
| Southwest Q4 2022 pre-tax negative impact (filed) | ∼$725–825 million |
| US Dept. of Transportation (DOT) penalty (Dec 2023) | $140 million |

*Largest DOT consumer-protection penalty in US aviation history.*

*“I fear that we are one thunderstorm,*

*one air traffic control event,*

*one IT router failure away*
*from a complete meltdown.”*

— Captain Casey Murray, President, Southwest Airlines Pilots Association (SWAPA)

Public podcast, November 7, 2022.

**Six weeks before the collapse.**

# Poll

The schedule collapsed and stranded two million passengers.

## Who is most responsible?

- **The engineers who built the scheduler**
- **The executives who approved it**
- **The regulators**
- **The staff who warned it was fragile**
- **No single person — the responsibility is shared**

**Vote now** — scan the code for your campus, *or* type the address.



**Melbourne**

pollev.com/juliangarcia



**Malaysia**

pollev.com/onghueyfang767

# Who is responsible?

**Who is responsible when an algorithm fails?**

**Discussion**

- The engineers who built the scheduling system in the 1990s?

*You'll write 150–250 words on this.*

# Who is responsible?

**Who is responsible when an algorithm fails?**

**Discussion**

- The engineers who built the scheduling system in the 1990s?
- The executives who deferred a documented modernisation budget?

*You'll write 150–250 words on this.*

# Who is responsible?

**Who is responsible when an algorithm fails?**

**Discussion**

- The engineers who built the scheduling system in the 1990s?
- The executives who deferred a documented modernisation budget?
- The regulators who didn't require resilience stress-testing?

*You'll write 150–250 words on this.*

# Who is responsible?

**Who is responsible when an algorithm fails?**

**Discussion**

- The engineers who built the scheduling system in the 1990s?
- The executives who deferred a documented modernisation budget?
- The regulators who didn't require resilience stress-testing?
- The pilots' union, which had publicly warned for years?

*You'll write 150–250 words on this.*

# Who is responsible?

**Who is responsible when an algorithm fails?**

**Discussion**

- The engineers who built the scheduling system in the 1990s?
- The executives who deferred a documented modernisation budget?
- The regulators who didn't require resilience stress-testing?
- The pilots' union, which had publicly warned for years?
- A system failure with no single owner — distributed across all of them?

*You'll write 150–250 words on this.*

# Before we argue

**How to start**

1. **30 seconds, silent.** Pick the position you find **hardest to defend** — and find one good reason for it anyway.
2. **60 seconds.** Turn to the person next to you and trade.

We'll surface the **tension** — not a verdict.

# What are we disagreeing about?

**Name it first**

- **Facts** — what actually happened?
- **Values** — what matters, and to whom?
- **Remedy** — what should change?

Two people can agree on the facts and still **disagree on the remedy**.

Local search

Hill climbing

Local minima

Random restarts

At scale

**In the lab**

# In the lab

# In the lab this week

*Pass tier.* **Aiming for a Pass**

1. Implement `count_violations` and `get_neighbours` for 8-queens (skeleton provided).
2. Implement hill climbing (write the loop from the pseudocode).
3. Run it from 50 random starts. Record solved vs stuck.
4. Plot violations-per-iteration for one solved run and one stuck run.
5. Hand-trace one hill-climbing step on your personalised board.
6. Write 150–250 words: “Who is responsible when an algorithm fails?”

*Credit tier.* **Aiming for a Credit** — all of the above, plus:

- Add **random restarts**. Plot success rate vs number of restarts.
- How many restarts get you above 95%?
- Try **sideways moves**: when no neighbour is better, accept an equal one (cap how many in a row). Does it escape the flat spots?
- Compare BFS, DFS, and greedy across different graph structures (see lab brief).



The system did exactly what it was built to do.

**That’s what makes “who’s responsible?” hard.**



The system did exactly what it was built to do.

**That's what makes "who's responsible?" hard.**

Next week: the same four algorithms,

**where the choice has consequences.**